Lecture 6: Parallel programming

Threads, channels and shared state

Willem Vanhulle

DevLab Rust 2025

Tuesday December 9, 2025

github.com/wvhulle/rust-course-ghent

Make sure you have already finished:

- session-5/tests/error-handling.rs

1.1. Plain Threads

```rust
use std::thread;
use std::time::Duration;

fn main() {
    thread::spawn(|| {
        for i in 0..10 {
            println!("Count in thread: {i}!");
            thread::sleep(Duration::from_millis(5));
        }
    });

    for i in 0..5 {
        println!("Main thread: {i}");
        thread::sleep(Duration::from_millis(5));
    }
}
```

(playground link)

What happens to the spawned thread?

1.1. Plain Threads

```rust
use std::thread;
use std::time::Duration;

fn main() {
    thread::spawn(|| {
        for i in 0..10 {
            println!("Count in thread: {i}!");
            thread::sleep(Duration::from_millis(5));
        }
    });

    for i in 0..5 {
        println!("Main thread: {i}");
        thread::sleep(Duration::from_millis(5));
    }
}
```

(playground link)

What happens to the spawned thread?

Terminated by operating system when main thread exits. No destructors called.

1.1. Plain Threads

```rust
thread::spawn(|| {
    for i in 0..10 {   }
});

for i in 0..5 {
    println!("Main thread: {i}");
    thread::sleep(Duration::from_millis(5));
}
```

(playground link)

How can we ensure the spawned thread finishes?

1.1. Plain Threads

```
1 thread::spawn(|| {
2     for i in 0..10 {   }
3 });
4
5 for i in 0..5 {
6     println!("Main thread: {i}");
7     thread::sleep(Duration::from_millis(5));
8 }                                                    (playground link)
```

How can we ensure the spawned thread finishes?

Use `JoinHandle` returned by `thread::spawn` and call `join()` on it.

```
1 fn main() {
2   let handle = thread::spawn(|| {
3     // thread code
4   });
5   // main thread code
6   handle.join();
7 }                                                    (playground link)
```

1.2. Receiving data from threads

How can we get a return value from a thread?

1.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```rust
fn main() {
    let handle = thread::spawn(|| {
        42
    });

    let result = handle.join().unwrap();
    println!("The answer is: {}", result);
}
```

(playground link)

What happens to the main thread if a spawned thread panics?

1.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```rust
fn main() {
    let handle = thread::spawn(|| {
        42
    });

    let result = handle.join().unwrap();
    println!("The answer is: {}", result);
}
```

(playground link)

What happens to the main thread if a spawned thread panics?

Nothing, the main thread continues executing.

1.3. Handling thread panics

If we don't join, we never see the panic.

1.3. Handling thread panics

If we don't join, we never see the panic.

If we don't handle the `Result` from `join()`, the panic is also ignored.

1.3. Handling thread panics

If we don't join, we never see the panic.

If we don't handle the `Result` from `join()`, the panic is also ignored.

We need to check the `Result` from `join()` to see if the thread panicked:

```
1  fn main() {
2      let handle = thread::spawn(|| {
3          panic!("Thread panicked!");
4      });
5      let result = handle.join();
6      match result {
7          Ok(_) => println!("Thread completed successfully."),
8          Err(e) => println!("Thread panicked: {:?}", e),
9      }
10 }
```

(playground link)

Warning

You are responsible for handling panics in spawned threads! Rust will not do this for you.

Exercise:

- examples/join-handle.rs

1.4. Sending references to threads

Not all references can be sent to threads.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    data.push(4); // modify while thread may be reading
    handle.join().unwrap();
}
```

(playground link)

What happens if a thread borrows data from the main thread?

1.4. Sending references to threads

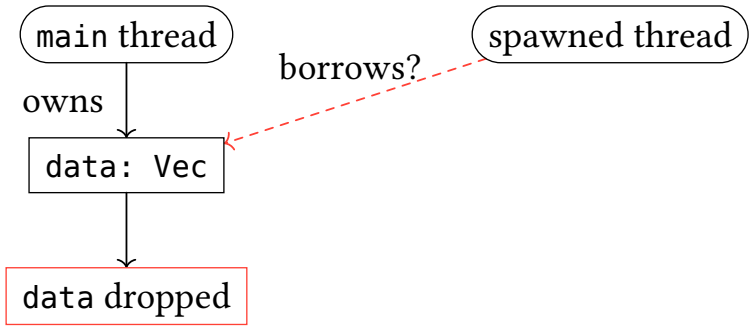
Not all references can be sent to threads.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    data.push(4); // modify while thread may be reading
    handle.join().unwrap();
}
```

(playground link)

What happens if a thread borrows data from the main thread?

Compilation error: closure may outlive the current function, but it borrows `data`.



1.4. Sending references to threads

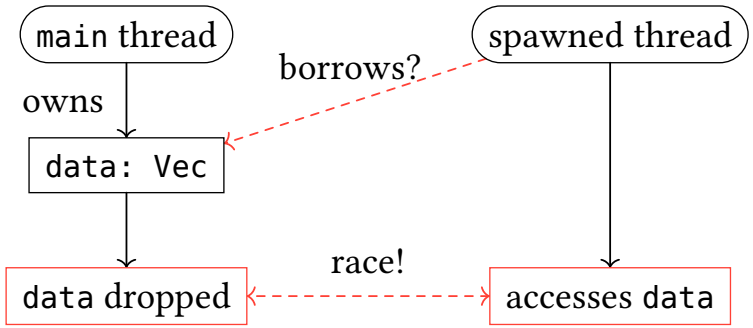
Not all references can be sent to threads.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    data.push(4); // modify while thread may be reading
    handle.join().unwrap();
}
```

(playground link)

What happens if a thread borrows data from the main thread?

Compilation error: closure may outlive the current function, but it borrows `data`.



1.5. Borrowing and threads (continued)

First attempt: ensure the thread finishes before `main` exits.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    handle.join().unwrap();
    println!("Original: {:?}", data); // try to use after thread
}
```

(playground link)

Does adding `join()` fix the borrow error?

1.5. Borrowing and threads (continued)

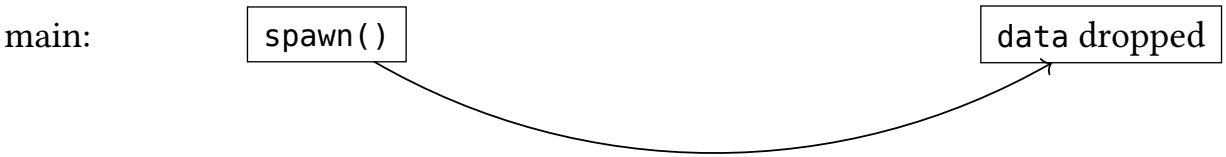
First attempt: ensure the thread finishes before `main` exits.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    handle.join().unwrap();
    println!("Original: {:?}", data); // try to use after thread
}
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

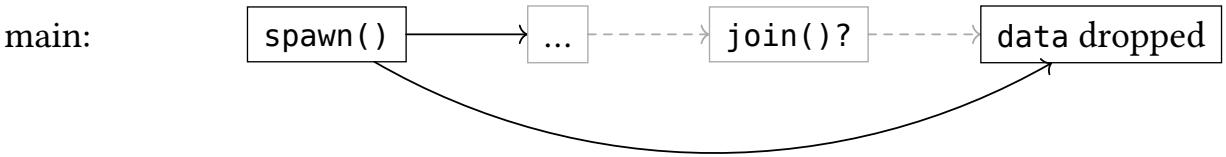
First attempt: ensure the thread finishes before `main` exits.

```
1 fn main() {
2     let data = vec![1, 2, 3];
3     let handle = thread::spawn(|| {
4         let sum: i32 = data.iter().sum();
5         println!("Sum: {sum}");
6     });
7     handle.join().unwrap();
8     println!("Original: {:?}", data); // try to use after thread
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

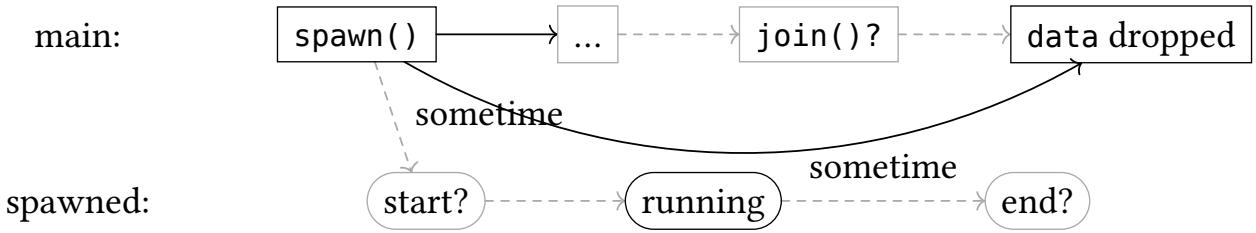
First attempt: ensure the thread finishes before `main` exits.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    handle.join().unwrap();
    println!("Original: {:?}", data); // try to use after thread
}
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

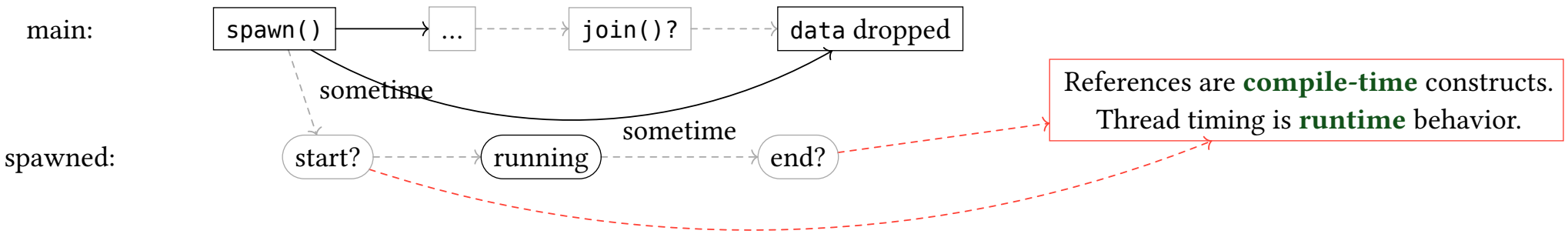
First attempt: ensure the thread finishes before `main` exits.

```rust
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        let sum: i32 = data.iter().sum();
        println!("Sum: {sum}");
    });
    handle.join().unwrap();
    println!("Original: {:?}", data); // try to use after thread
}
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.6. Scoped threads

The problem: `thread::spawn` requires `'static` because the thread may outlive the caller.

```
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        println!("Data: {:?}", data); // error: `data` does not live long enough
    });
    handle.join().unwrap();
}
```

(playground link)

1.6. Scoped threads

The problem: `thread::spawn` requires `'static` because the thread may outlive the caller.

```
fn main() {
    let data = vec![1, 2, 3];
    let handle = thread::spawn(|| {
        println!("Data: {:?}", data); // error: `data` does not live long enough
    });
    handle.join().unwrap();
}
```

(playground link)

Solution: `thread::scope` guarantees all spawned threads complete before the scope ends.

```
fn main() {
    let data = vec![1, 2, 3];
    thread::scope(|s| {
        s.spawn(|| println!("Data: {:?}", data)); // borrowing works!
    }); // all threads joined here automatically
    println!("Back in main: {:?}", data);
}
```

(playground link)

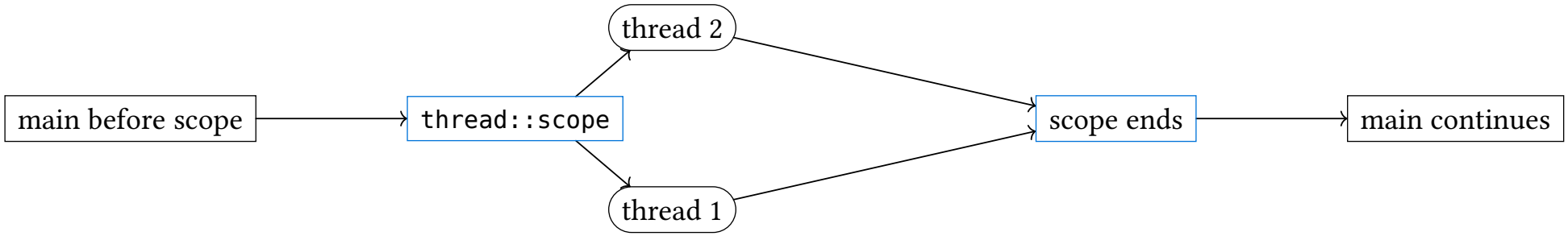
1.7. Multiple scoped threads

```rust
fn main() {
    let mut data = vec![1, 2, 3];

    thread::scope(|s| {
        s.spawn(|| println!("Thread 1: {:?}", data)); // shared borrow
        s.spawn(|| println!("Thread 2: {:?}", data)); // shared borrow
    });

    data.push(4); // mutable access after scope ends
}
```

(playground link)



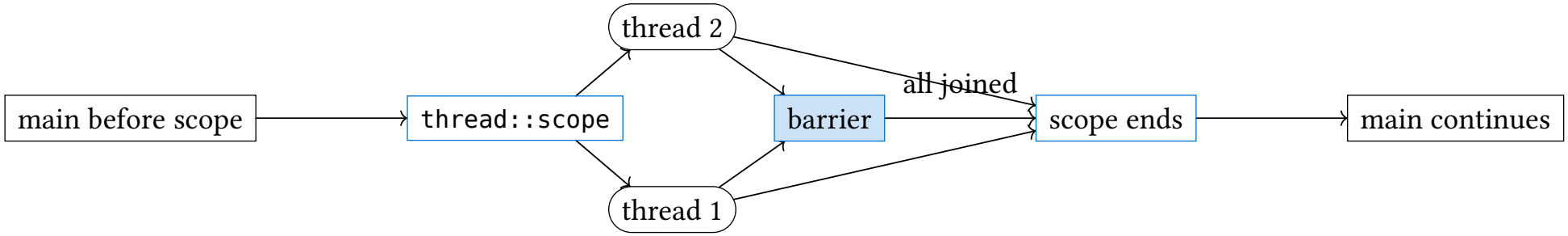
1.7. Multiple scoped threads

```rust
fn main() {
    let mut data = vec![1, 2, 3];

    thread::scope(|s| {
        s.spawn(|| println!("Thread 1: {:?}", data)); // shared borrow
        s.spawn(|| println!("Thread 2: {:?}", data)); // shared borrow
    });

    data.push(4); // mutable access after scope ends
}
```

(playground link)



1.8. Moving to threads

In practice, scoped threads are not used very often.

The most common use case is to move ownership of data to the thread using the `move` keyword to transfer ownership at spawn time

```rust
fn main() {
    let data = String::from("hello");
    let handle = thread::spawn(move || {
        println!("Data: {data}"); // `data` moved into closure, now owned by thread
    });
    handle.join().unwrap();
}
```

(playground link)

Move can also be used for blocks (not closures):

1.9. Review: what is `'static`?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is `'static` if it contains no non-`'static` references.*

Things that are `'static`:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- `'static` references: `&'static str`, `&'static T`

Things that are not `'static`:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not `'static`
- Types with non-`'static` fields: `&T`, `&'a T`, `Vec<&T>`, etc.

1.9. Review: what is `'static`?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is `'static` if it contains no non-`'static` references.*

Things that are `'static`:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- `'static` references: `&'static str`, `&'static T`

Things that are not `'static`:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not `'static`
- Types with non-`'static` fields: `&T`, `&'a T`, `Vec<&T>`, etc.

Strings are owned and without references, so they are `'static` and can be moved

1.9. Review: what is `'static`?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is `'static` if it contains no non-`'static` references.*

Things that are `'static`:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- `'static` references: `&'static str`, `&'static T`

Things that are not `'static`:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not `'static`
- Types with non-`'static` fields: `&T`, `&'a T`, `Vec<&T>`, etc.

Strings are owned and without references, so they are `'static` and can be moved

1.9.1. Spawn function

The standard library spawn function looks like this (simplified):

```
1 fn spawn<F, T>(f: F) -> JoinHandle<T>
2 where
3     F: FnOnce() -> T + 'static,
4     T: 'static                                    (playground link)
```

(Similar for asynchronous spawns like `tokio::spawn`, see next session.)

2.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

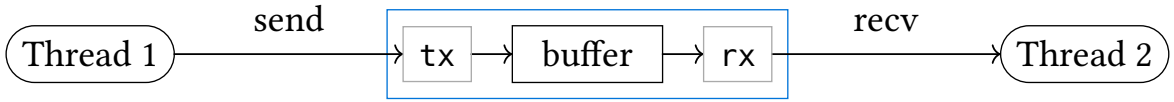
- **Sender** (`tx`): pushes messages into the buffer
- **Receiver** (`rx`): pulls messages from the buffer

2.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (`tx`): pushes messages into the buffer
- **Receiver** (`rx`): pulls messages from the buffer

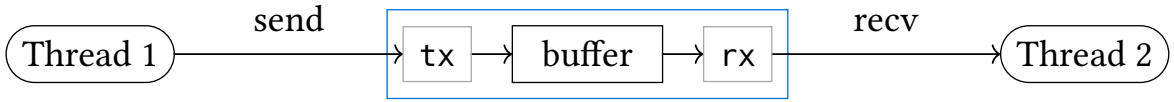


2.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (`tx`): pushes messages into the buffer
- **Receiver** (`rx`): pulls messages from the buffer



Key properties:

- **Decouples** sender and receiver (no shared mutable state)
- **Asynchronous**: sender doesn't wait for receiver (unless bounded)
- **Ownership transfer**: messages are moved, not shared

```rust
fn main() {
    let (tx, rx) = std::sync::mpsc::channel();
    std::thread::spawn(move || {
        let thread_id = std::thread::current().id();
        for i in 0..10 {
            tx.send(format!("Message {i}")).unwrap();
            println!("{thread_id:?}: sent Message {i}");
        }
        println!("{thread_id:?}: done");
    });
    std::thread::sleep(std::time::Duration::from_millis(100));

    for msg in rx.iter() {
        println!("Main: got {msg}");
    }
}
```

(playground link)

In an unbounded channel:

- `send()` never blocks (a kind of "asynchronous" behavior)
- returns a `Result::Err` when all receivers have been dropped

Definition 2.2.1. *A channel is "closed" when all receivers have been dropped.*

2.3. Bounded channels

If you don't want the channel buffer to grow indefinitely and cause memory overflow, you can use a bounded channel:

2.3. Bounded channels

If you don't want the channel buffer to grow indefinitely and cause memory overflow, you can use a bounded channel:

```rust
fn main() {
    let (tx, rx) = mpsc::sync_channel(3);

    thread::spawn(move || {
        let thread_id = thread::current().id();
        for i in 0..10 {
            tx.send(format!("Message {i}")).unwrap();
            println!("{thread_id:?}: sent Message {i}");
        }
        println!("{thread_id:?}: done");
    });
    thread::sleep(Duration::from_millis(100));

    for msg in rx.iter() {
        println!("Main: got {msg}");
    }
}
```

(playground link)

This channel provides an additional method `try_send()` that returns immediately with an error if the buffer is full.

2.4. Exercises

- `examples/channel.rs`

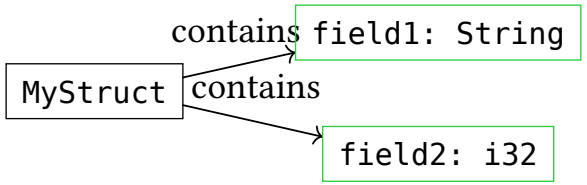
3.1. Marker Traits

What are auto traits?

3.1. Marker Traits

What are auto traits?

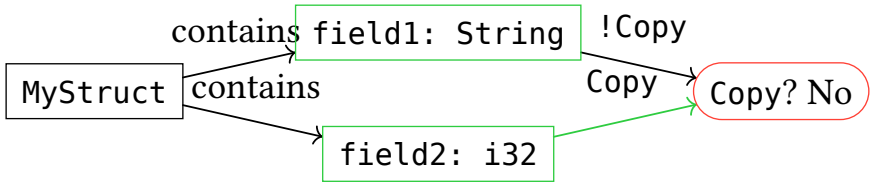
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



3.1. Marker Traits

What are auto traits?

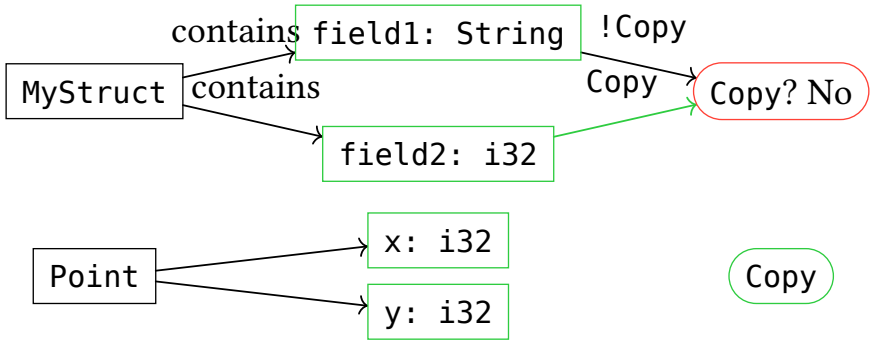
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



3.1. Marker Traits

What are auto traits?

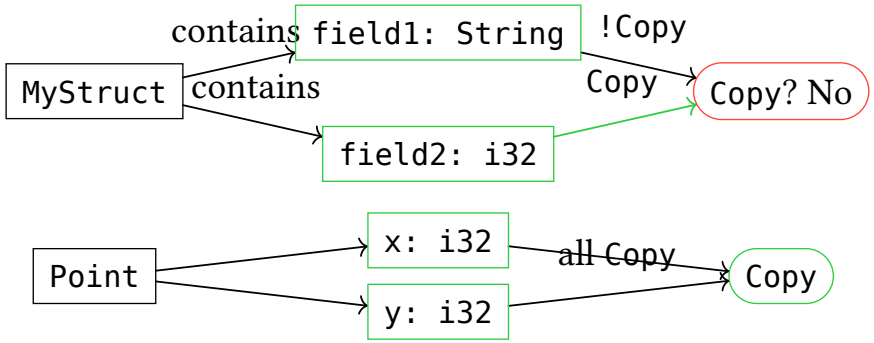
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



3.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.

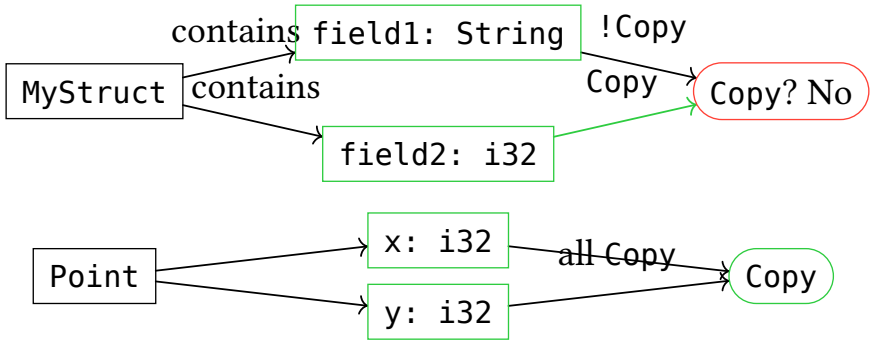


What are `unsafe` traits?

3.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



What are `unsafe` traits?

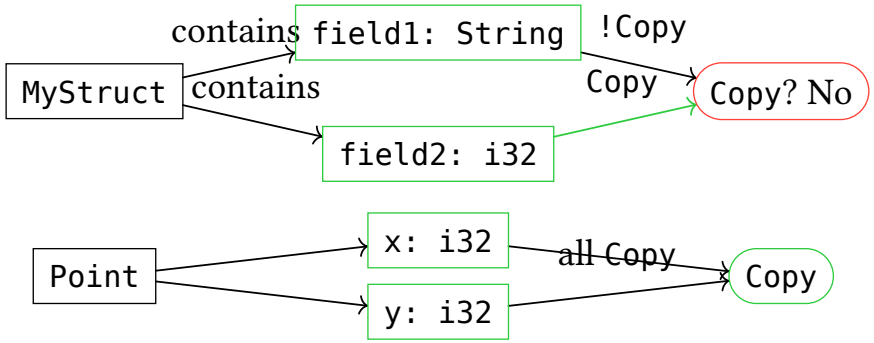
Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement `unsafe` traits manually when you know it is valid.

3.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



What are `unsafe` traits?

Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement `unsafe` traits manually when you know it is valid.

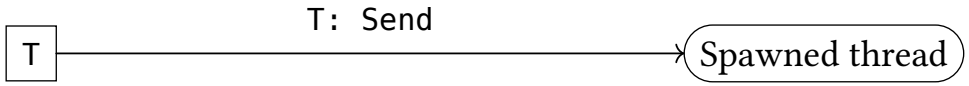
Warning

Implementing `unsafe` traits requires `unsafe` blocks. Using them as constraints does not.

3.2. Send and Sync

Definition 3.2.1.

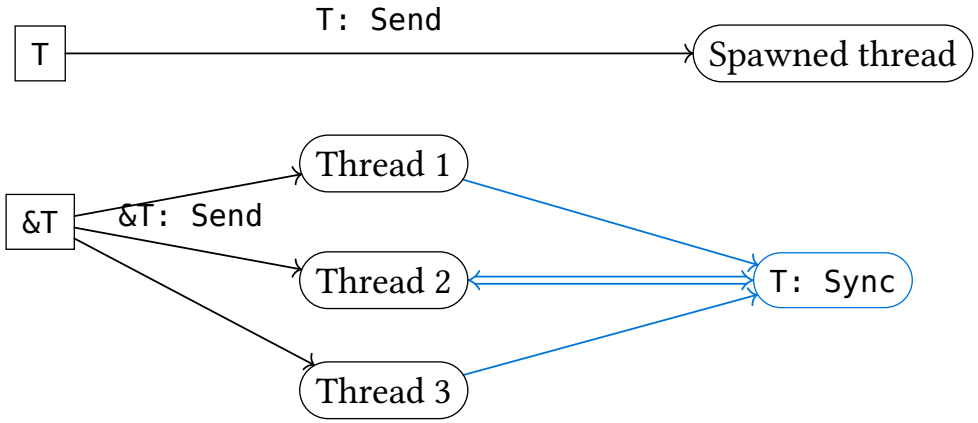
- *`Send`: safe to **move** `T` to another thread*
- *`Sync`: safe to **share** `&T` across threads (i.e., `&T: Send`)*



3.2. Send and Sync

Definition 3.2.1.

- *Send: safe to **move** `T` to another thread*
- *Sync: safe to **share** `&T` across threads (i.e., `&T: Send`)*



Info

Send and Sync are unsafe auto traits.

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability**: `i32`, `bool`, `String`, `Vec<T>`

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability**: `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership**: `Arc<T>` (atomic ref counting)

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability**: `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership**: `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics**: `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

3.3. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability**: `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership**: `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics**: `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

When the type parameters `T` are Send + Sync.

3.4. Atomics

Atomics are low-level types for lock-free concurrent programming.

3.4. Atomics

Atomics are low-level types for lock-free concurrent programming.

Two problems with normal memory operations:

- Compilers and CPUs may **reorder** instructions for optimization
- Read-modify-write (e.g., `x += 1`) is **not atomic** (multiple CPU instructions)

3.4. Atomics

Atomics are low-level types for lock-free concurrent programming.

Two problems with normal memory operations:

- Compilers and CPUs may **reorder** instructions for optimization
- Read-modify-write (e.g., `x += 1`) is **not atomic** (multiple CPU instructions)

```rust
static mut DATA: u32 = 0;
static mut READY: bool = false;

fn thread1() { unsafe { DATA = 42; READY = true; } }
fn thread2() { unsafe { while !READY {} println!("{}", DATA); } } // might print 0!
```

(playground link)

CPU might execute `READY = true` before `DATA = 42`.

3.5. Atomics (continued)

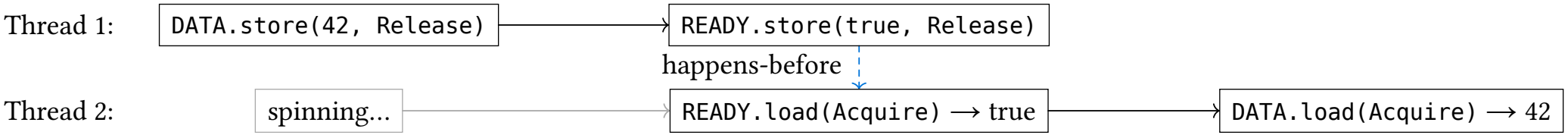
Solution: atomic operations with memory ordering guarantees.

```rust
use std::sync::atomic::{AtomicBool, AtomicU32, Ordering};

static DATA: AtomicU32 = AtomicU32::new(0);
static READY: AtomicBool = AtomicBool::new(false);

fn thread1() {
    DATA.store(42, Ordering::Release);
    READY.store(true, Ordering::Release);
}
fn thread2() {
    while !READY.load(Ordering::Acquire) {}
    println!("{}", DATA.load(Ordering::Acquire)); // guaranteed 42
}
```

(playground link)



The happens-before edge is on `READY` (same atomic, Release-store ⟶ Acquire-load).

This transitively makes `DATA.store` visible to `DATA.load`.

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

- **Always supported**: `AtomicBool`, `AtomicU8`, `AtomicU16`, `AtomicU32`, `AtomicUsize`
- **64-bit only**: `AtomicU64` (not available on 32-bit targets)
- **Rare**: `AtomicU128` (only some x86-64 CPUs)

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

- **Always supported**: `AtomicBool`, `AtomicU8`, `AtomicU16`, `AtomicU32`, `AtomicUsize`
- **64-bit only**: `AtomicU64` (not available on 32-bit targets)
- **Rare**: `AtomicU128` (only some x86-64 CPUs)

Check at compile time:

```
1 #[cfg(target_has_atomic = "64")]
2 use std::sync::atomic::AtomicU64;                    (playground link)
```

3.6. Atomics: hardware support

Not all CPUs support all atomic operations natively.

- **Always supported**: `AtomicBool`, `AtomicU8`, `AtomicU16`, `AtomicU32`, `AtomicUsize`
- **64-bit only**: `AtomicU64` (not available on 32-bit targets)
- **Rare**: `AtomicU128` (only some x86-64 CPUs)

Check at compile time:

```rust
#[cfg(target_has_atomic = "64")]
use std::sync::atomic::AtomicU64;
```

(playground link)

If hardware doesn't support an atomic size, Rust either:

- Won't compile (type not available)
- Falls back to a lock-based implementation (slower)

Info

On modern x86-64 and ARM64, all common atomics are hardware-supported.

O'REILLY®

Rust Atomics and Locks

Low-Level Concurrency in Practice

Mara Bos

3.7. Send + !Sync

These types can be moved to other threads, but cannot be shared via `&T`:

3.7. Send + !Sync

These types can be moved to other threads, but cannot be shared via `&T`:

- **Non-thread-safe interior mutability**: `Cell<T>`, `RefCell<T>`
  - Allow mutation through `&T` without synchronization
  - Safe to own exclusively on one thread, unsafe to share references

3.8. !Send + Sync

These types are safe to access (via shared references) from multiple threads, but cannot be moved to another thread:

- **Lock guards**: `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`
  - Must be unlocked on the thread that acquired the lock (POSIX requirement)

Proposition 3.8.1. *`MutexGuard<T: Sync>: !Send + Sync`*

3.9. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

3.9. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

- dereferences and has non-atomic count: `Rc<T>`
- considered unsafe in general `*const T`, `*mut T`

3.10. Summary

	Sync	!Sync

Send	`i32`, `bool`, `String`, `Vec<T>`
	`Arc<T>`, `Mutex<T>`, `Atomic*`
	Safe to move and share
		`Cell<T>`, `RefCell<T>`
		Interior mutability: safe to move, not share

!Send	`MutexGuard`, `RwLock*Guard` *Thread-bound: safe to share, not move*	`Rc<T>`, `*const T`, `*mut T` *Not thread-safe at all*

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)

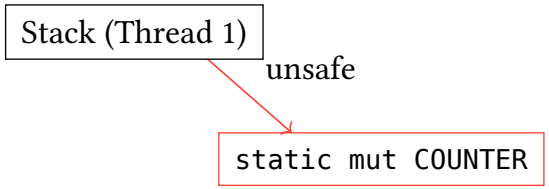
`static mut COUNTER`

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)

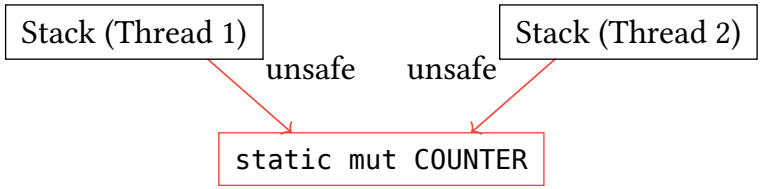


4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)

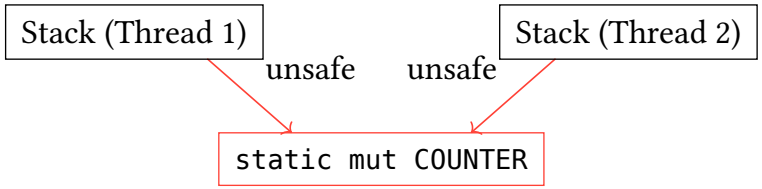


4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)

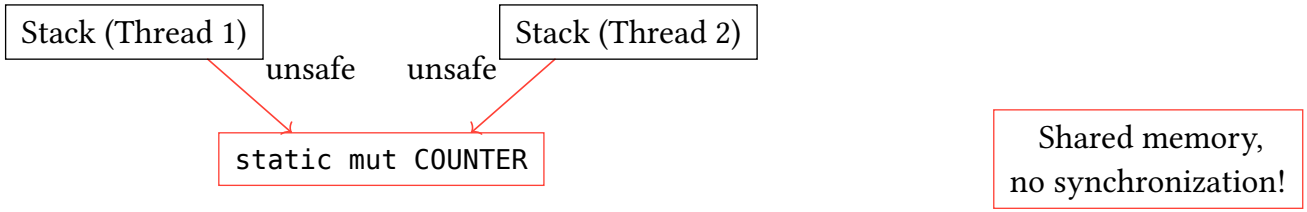


4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)



Disadvantages:

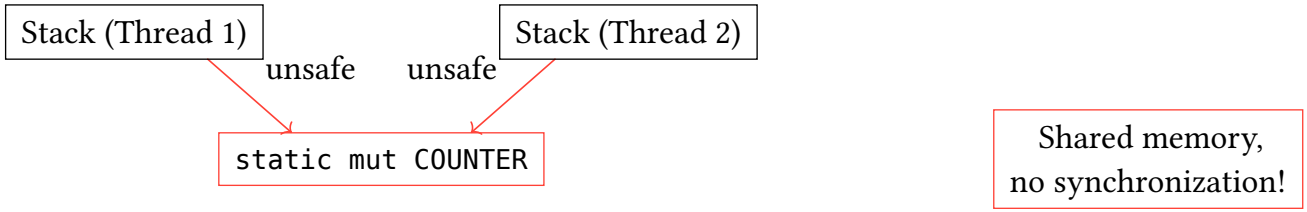
- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)



Disadvantages:

- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

Solution:

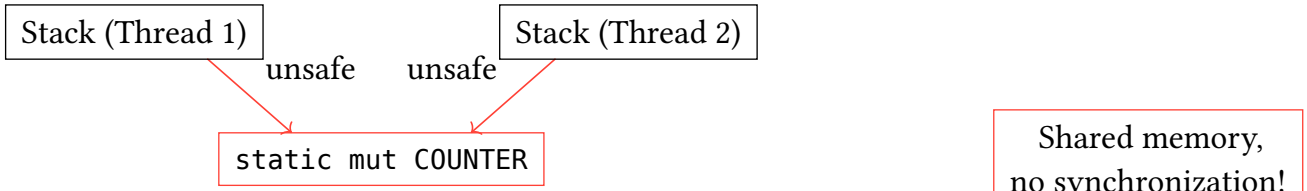
1. Use reference counted variable (next slides)

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```rust
static mut COUNTER: u32 = 0;
fn increment() {
    unsafe {
        COUNTER += 1;
    }
}
```

(playground link)



Disadvantages:

- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

Solution:

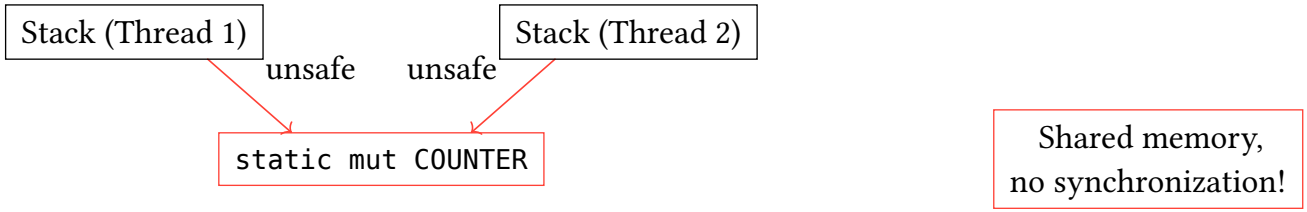
1. Use reference counted variable (next slides)
2. Use a lock to ensure exclusive access (later)

4.1. Sharing data

The unsafe way to share data between threads is to use `static mut`:

```
1 static mut COUNTER: u32 = 0;
2 fn increment() {
3     unsafe {
4         COUNTER += 1;
5     }
6 }
```

(playground link)



Disadvantages:

- Unsafe: requires `unsafe` blocks to access
- Needs to be initialized for the entire program lifetime

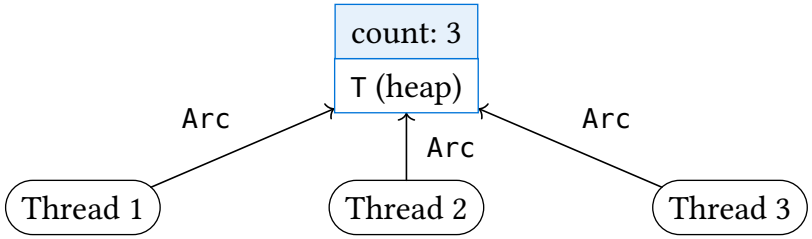
Solution:

1. Use reference counted variable (next slides)
2. Use a lock to ensure exclusive access (later)

(Notice that in this simple case we could just use an `AtomicU32` instead. This is just an example.)

4.2. Concept

Definition 4.2.1. `Arc<T>` = ***Atomic Reference Counted*** *pointer for shared ownership across threads.*



Key properties:

- `Arc::clone(&v)` increments ref count (atomic operation)
- When count reaches 0, data is dropped
- `Arc<T>: Send + Sync` when `T: Send + Sync`

4.2. Concept

4.2.1. Arc example

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let handles: Vec<_> = (0..3).map(|i| {
        let data = Arc::clone(&data);
        thread::spawn(move || {
            println!("Thread {i}: {:?}", data);
        })
    }).collect();

    for h in handles { h.join().unwrap(); }
}
```

(playground link)

4.2. Concept

4.2.1. Arc example

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let handles: Vec<_> = (0..3).map(|i| {
        let data = Arc::clone(&data);
        thread::spawn(move || {
            println!("Thread {i}: {:?}", data);
        })
    }).collect();

    for h in handles { h.join().unwrap(); }
}
```

(playground link)

Who drops the data?

4.2.1. Arc example

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let handles: Vec<_> = (0..3).map(|i| {
        let data = Arc::clone(&data);
        thread::spawn(move || {
            println!("Thread {i}: {:?}", data);
        })
    }).collect();

    for h in handles { h.join().unwrap(); }
}
```

(playground link)

Who drops the data?

The last thread to finish (last `Arc` clone to go out of scope).

Can we mutate `T` through `Arc<T>`?

4.2.1. Arc example

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let handles: Vec<_> = (0..3).map(|i| {
        let data = Arc::clone(&data);
        thread::spawn(move || {
            println!("Thread {i}: {:?}", data);
        })
    }).collect();

    for h in handles { h.join().unwrap(); }
}
```

(playground link)

Who drops the data?

The last thread to finish (last `Arc` clone to go out of scope).

Can we mutate `T` through `Arc<T>`?

No, `Arc` only provides shared access. Use `Arc<Mutex<T>>` for mutation.

5.1. Mutex

Prevents race conditions on complex shared data using synchronisation.

How are you used to do data synchronisation in your favourite language?

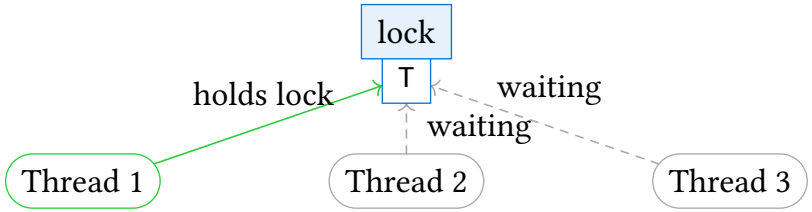
5.1. Mutex

Prevents race conditions on complex shared data using synchronisation.

How are you used to do data synchronisation in your favourite language?

...

Definition 5.1.1. `Mutex<T>` = ***Mutual Exclusion*** *lock for exclusive access to* `T` *across threads.*



Key operations:

- `lock()` ⟶ blocks until lock acquired, returns `MutexGuard<T>`
- `MutexGuard` auto-releases lock when dropped (RAII)
- `Mutex<T>: Sync` when `T: Send`

5.1. Mutex

5.1.1. Mutex example

```rust
use std::sync::Mutex;

fn main() {
    let counter = Mutex::new(0);

    {
        let mut guard = counter.lock().unwrap();
        *guard += 1;
    } // lock released here

    println!("Counter: {:?}", counter.lock().unwrap());
}
```

(playground link)

5.1. Mutex

5.1.1. Mutex example

```rust
use std::sync::Mutex;

fn main() {
    let counter = Mutex::new(0);

    {
        let mut guard = counter.lock().unwrap();
        *guard += 1;
    } // lock released here

    println!("Counter: {:?}", counter.lock().unwrap());
}
```

(playground link)

What is a `PoisonError`?

5.1. Mutex

5.1.1. Mutex example

```rust
use std::sync::Mutex;

fn main() {
    let counter = Mutex::new(0);

    {
        let mut guard = counter.lock().unwrap();
        *guard += 1;
    } // lock released here

    println!("Counter: {:?}", counter.lock().unwrap());
}
```

(playground link)

What is a `PoisonError`?

Another thread panicked while holding the lock. The data may be in an inconsistent state.

Why does `Mutex<T>` require `T: Send` but not `T: Sync`?

5.1. Mutex

5.1.1. Mutex example

```rust
use std::sync::Mutex;

fn main() {
    let counter = Mutex::new(0);

    {
        let mut guard = counter.lock().unwrap();
        *guard += 1;
    } // lock released here

    println!("Counter: {:?}", counter.lock().unwrap());
}
```

(playground link)

What is a `PoisonError`?

Another thread panicked while holding the lock. The data may be in an inconsistent state.

Why does `Mutex<T>` require `T: Send` but not `T: Sync`?

The mutex guarantees only one thread accesses `T` at a time, so `T` doesn't need to support concurrent access.

5.1. Mutex

5.1.2. Arc + Mutex: shared mutable state

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));

    let handles: Vec<_> = (0..4).map(|_| {
        let counter = Arc::clone(&counter);
        thread::spawn(move || {
            let mut guard = counter.lock().unwrap();
            *guard += 1;
        })
    }).collect();

    for h in handles { h.join().unwrap(); }
    println!("Counter: {}", counter.lock().unwrap()); // 4
}
```

(playground link)

- `Arc` provides shared ownership across threads
- `Mutex` provides exclusive mutable access

5.2. Interior mutability

Definition 5.2.1 (Interior mutability). *A type that allows mutation through a shared reference (`&T` → `&mut` inner).*

5.2. Interior mutability

Definition 5.2.1 (Interior mutability). *A type that allows mutation through a shared reference (`&T` → `&mut inner`).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
`Cell<T>`	No	Compile-time	Simple values, `Copy` types
`RefCell<T>`	No	Runtime (panics)	Complex borrows, single-threaded
`Mutex<T>`	Yes	Runtime (blocks)	Exclusive access across threads
`RwLock<T>`	Yes	Runtime (blocks)	Many readers, few writers
`Atomic*`	Yes	Hardware	Counters, flags, lock-free

5.2. Interior mutability

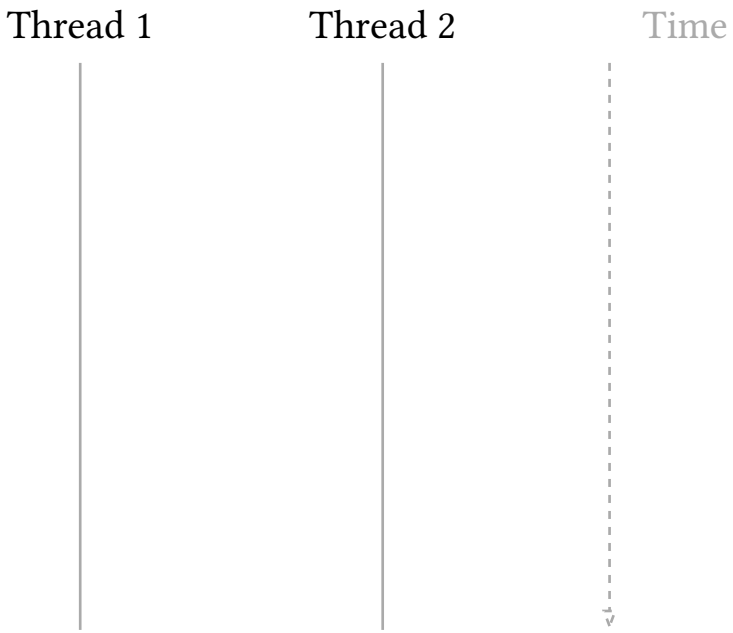
Definition 5.2.1 (Interior mutability). *A type that allows mutation through a shared reference (`&T` → `&mut inner`).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
`Cell<T>`	No	Compile-time	Simple values, `Copy` types
`RefCell<T>`	No	Runtime (panics)	Complex borrows, single-threaded
`Mutex<T>`	Yes	Runtime (blocks)	Exclusive access across threads
`RwLock<T>`	Yes	Runtime (blocks)	Many readers, few writers
`Atomic*`	Yes	Hardware	Counters, flags, lock-free

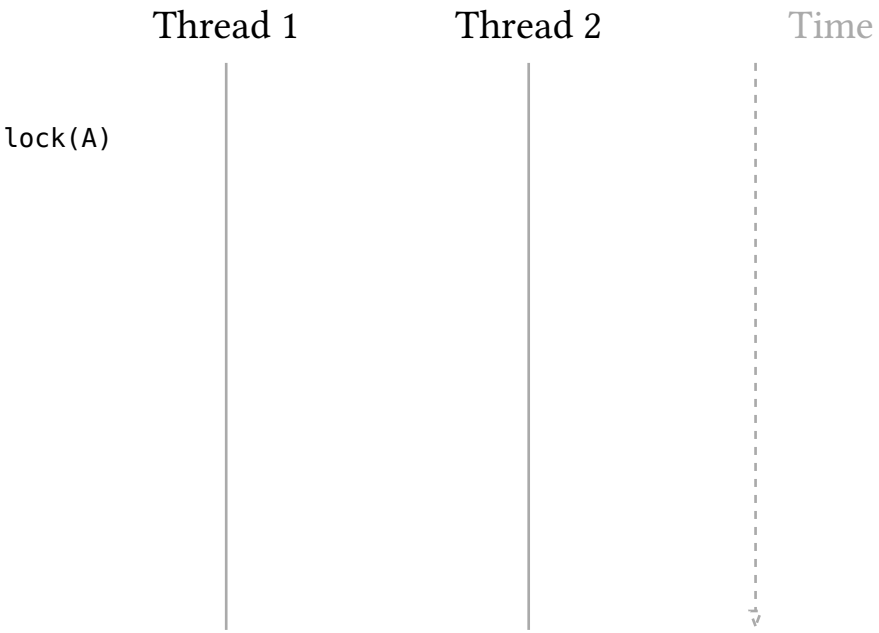
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



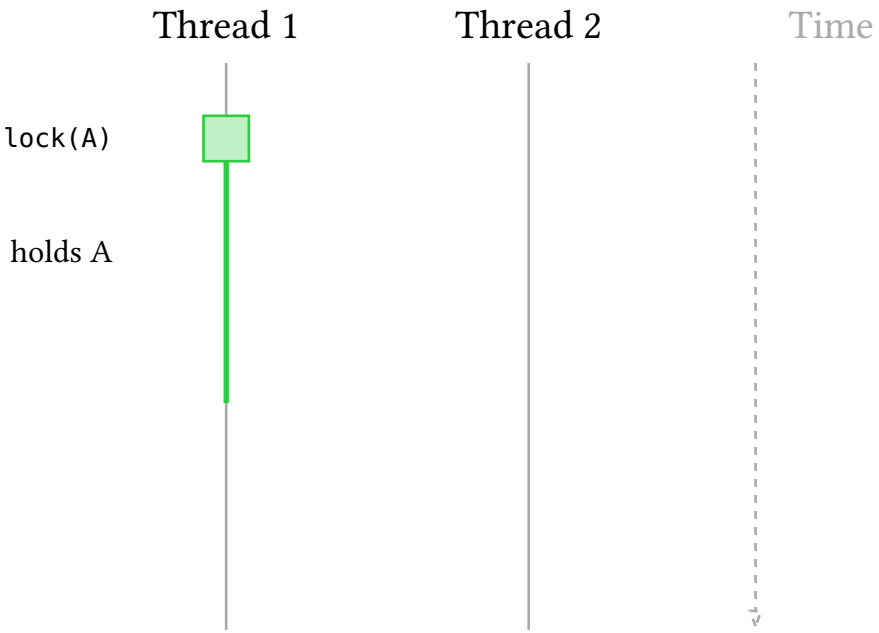
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



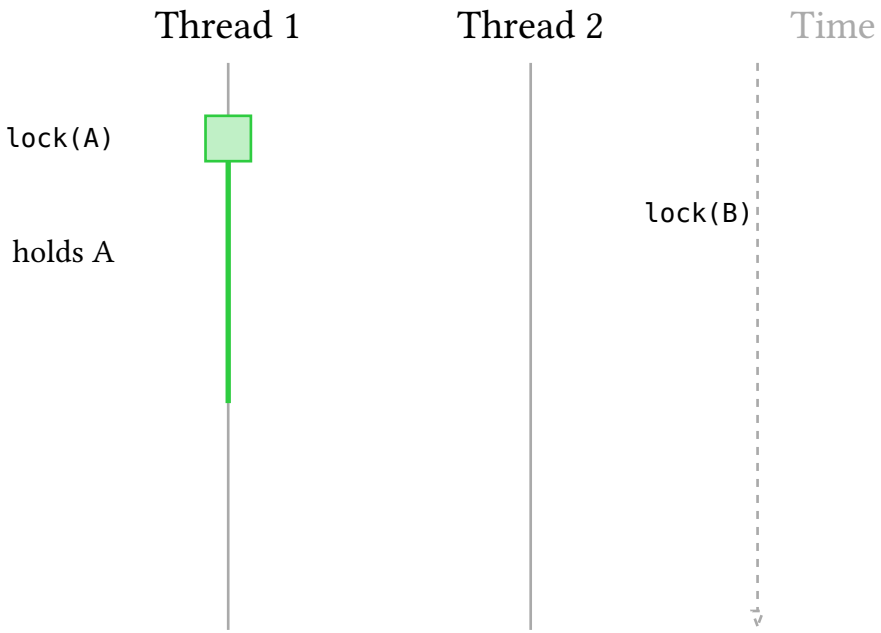
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



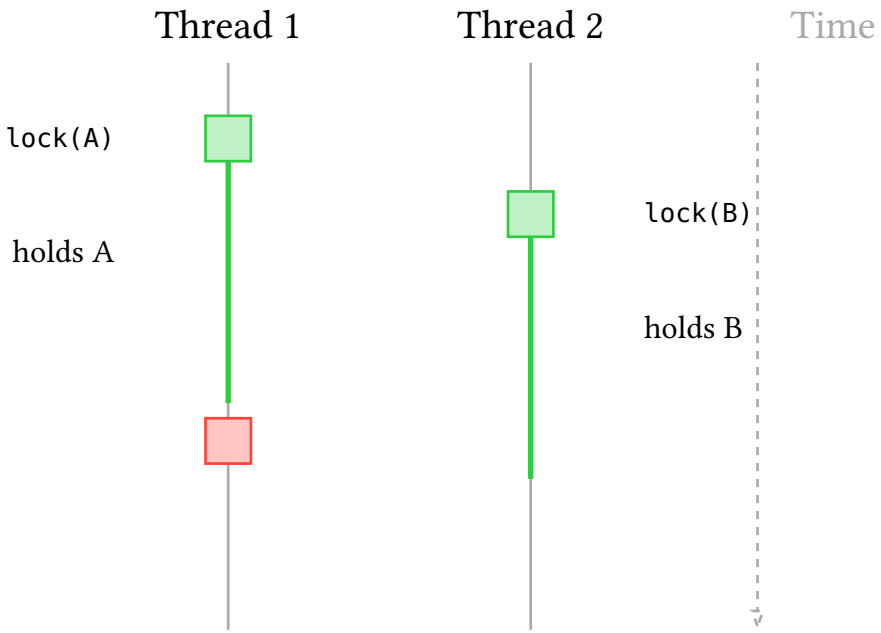
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



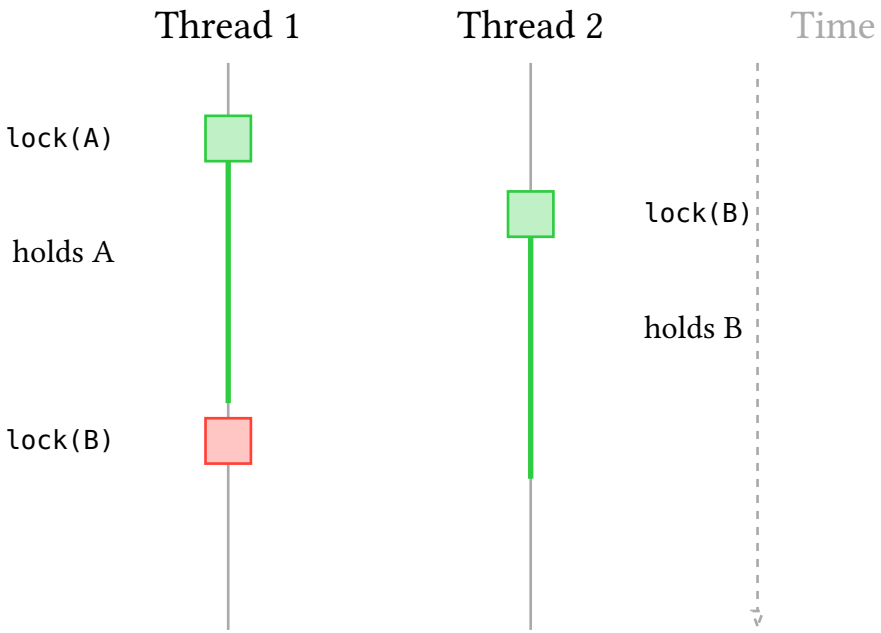
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



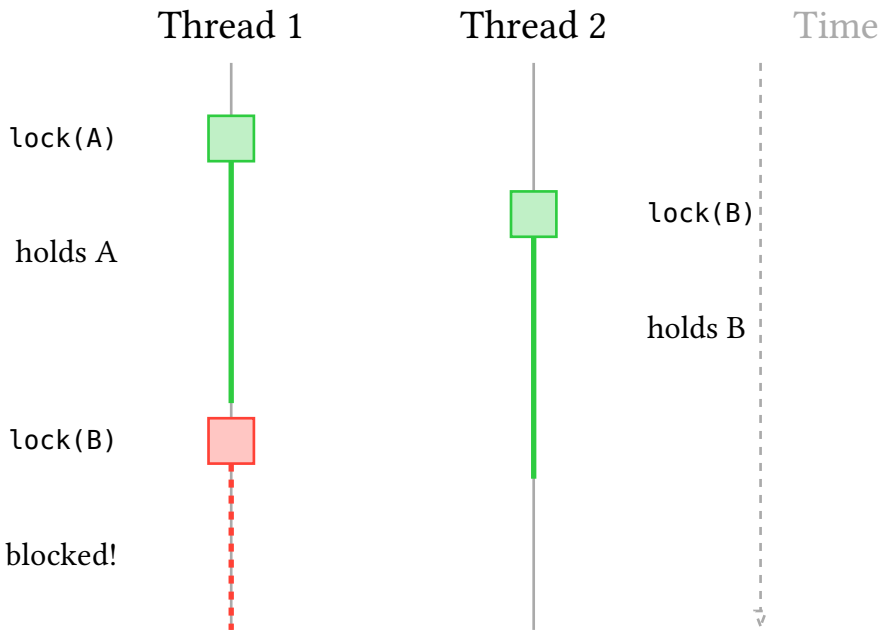
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



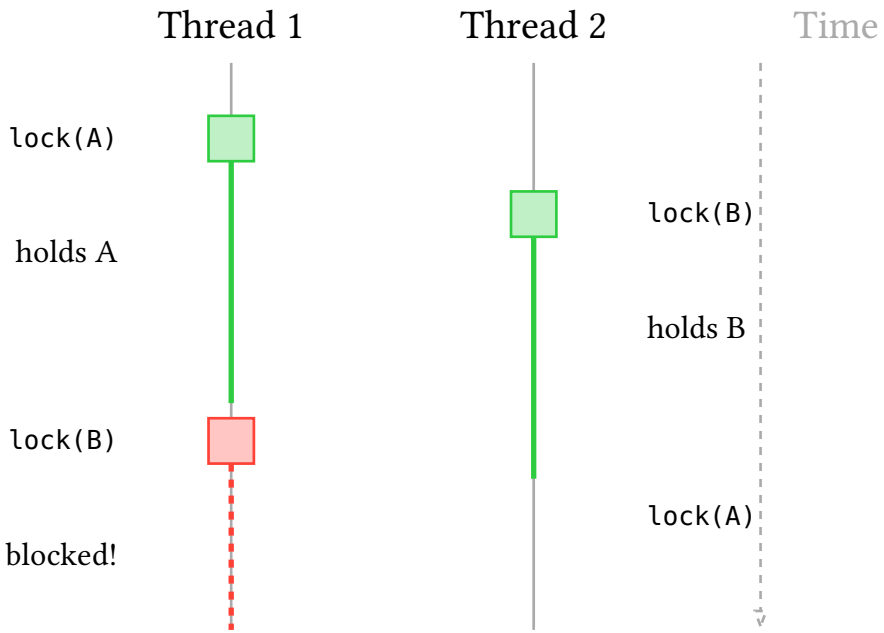
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



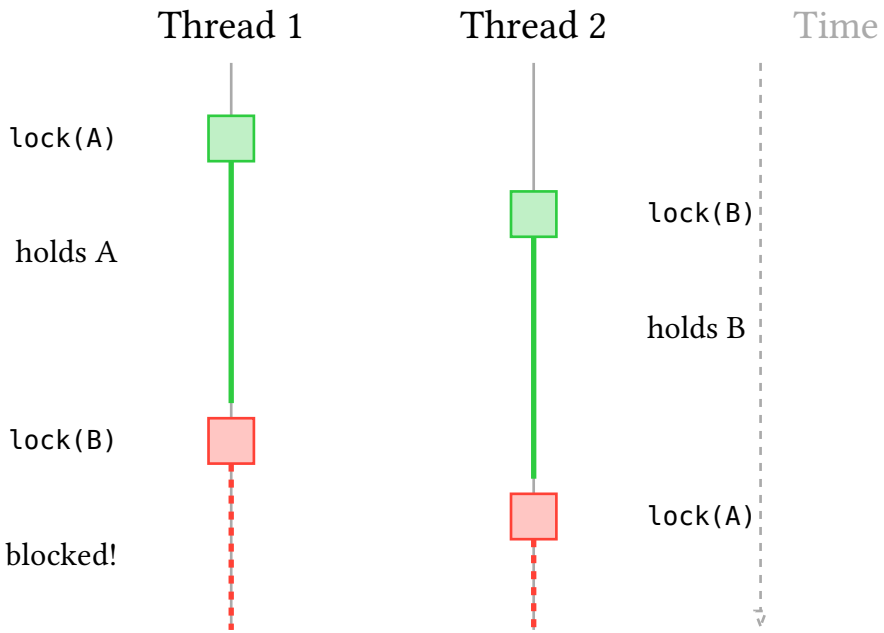
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



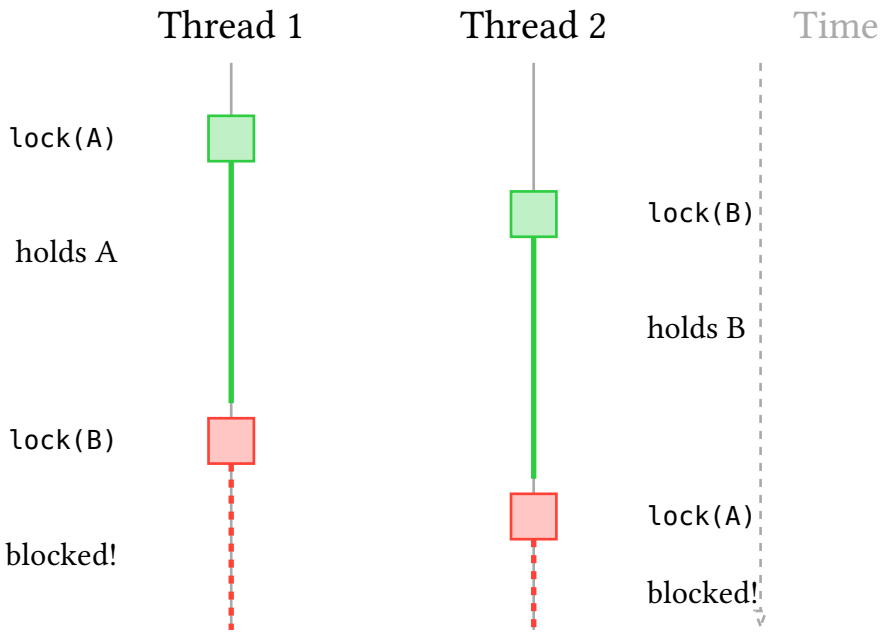
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



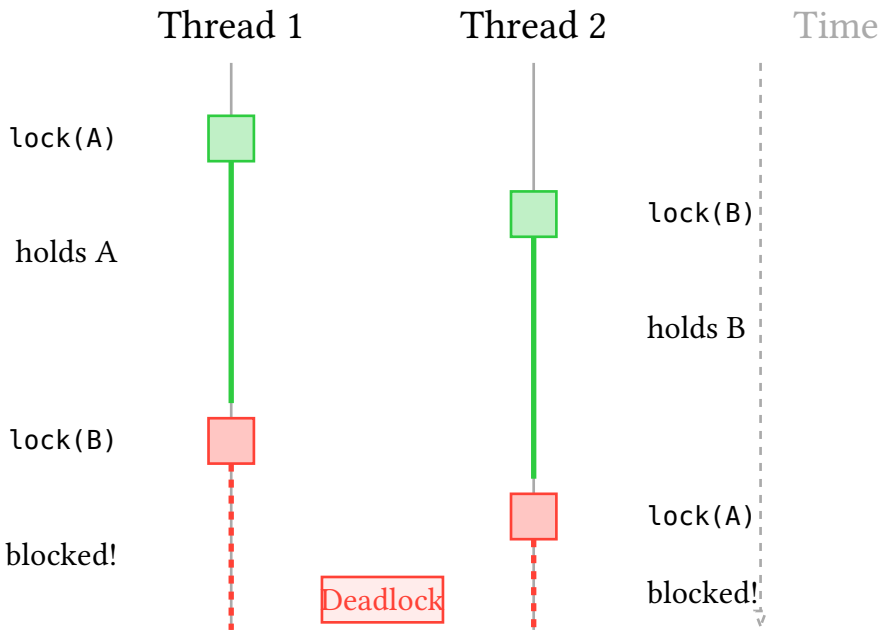
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



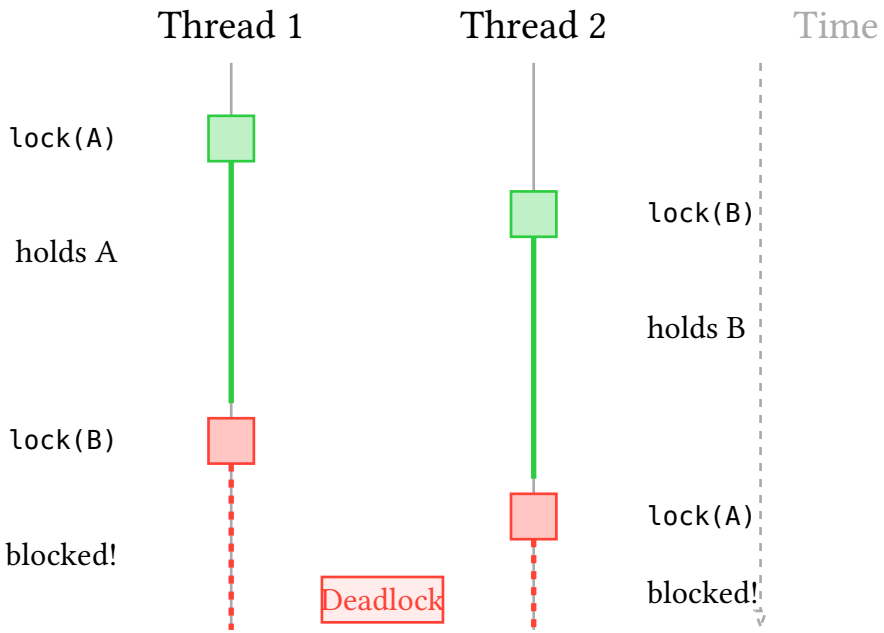
5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



5.3. Deadlocks

The `Arc<Mutex<T>>` pattern is common but has drawbacks: forgetting to drop the guard before blocking operations



Exercise:

- examples/philosophers.rs

Solutions are provided as `*-solution.rs` files.

5.4. Best practices for Mutexes

Structure code to minimize shared mutable state in mutexes.

Alternatives:

- Use `RwLock<T>` for many readers, few writers
- Use `Atomic*` types for simple counters/flags

If really necessary:

- Use channels for **one-directional flow** (but they are also hard to maintain)

One more exercise:

- `examples/link.rs` (you need to have openssl installed, use the `flake.nix` or your package manager)

Solutions are provided as `*-solution.rs` files.